

Polyglot / Steganaliz Servisi — 3. Hafta Raporu (Gün 11-15)

Dönem: 31 Ağustos - 5 Eylül 2026 **Kapsam:** Web API (FastAPI) ve Arka Plan Servis Mimarisi

1. Yönetici Özeti

3. Hafta kapsamında, 1-2. Hafta'da CLI script'leri olarak inşa edilen steganaliz motoru (`detect_trailer` + `size_analysis` + `entropy` + `extract` + `video_metadata`) bir **FastAPI REST servisine** dönüştürüldü. Boş bir uygulama iskeletinden (Gün 11) başlanıp; dosya yükleme + iki aşamalı doğrulama (Content-Type beyanı + magic bytes) eklenip (Gün 12); CPU-yoğun analiz `asyncio.to_thread` ile event loop'u bloklamadan pipeline'a bağlanıp, ayıklanan video `StaticFiles` ile statik olarak sunulup (Gün 13); nihayetinde üç sinyalin (trailer/entropy/boyut sapması) ağırlıklı birleşimiyle 0-100 aralığında bir `threat_score` ve dinamik bir `analysis_summary` üretilerek (Gün 14) plan'ın istediği `AnalyzeResponse` şeması tamamlandı. Hafta, tüm hata senaryolarının (400/413/422/500) anlamlı JSON mesajlarıyla ele alındığı, global bir exception handler ile öngörülmemiş hataların da stack-trace sızdırmadan yakalandığı bir hata yönetimi turuyla (Gün 15) kapandı.

Beş günün (Gün 11-15) tamamı planlandığı gibi tamamlandı. Bu raporun hazırlanması için canlı bir `uvicorn` sunucusu ayağa kaldırılıp Gün 11-15'in tüm test senaryoları (health-check, başarılı/başarısız yükleme, polyglot/temiz dosya analizi, hata kodları) yeniden çalıştırılarak bağımsız bir doğrulama turundan geçirildi (bkz. Bölüm 9). Herhangi bir eksik veya açık madde kalmadı.

2. Hafta 3 Hedefi

Proje planına göre bu haftanın çıktısı şu şekilde tanımlanmıştır:

"Dışarıdan sorgulanabilir, tam fonksiyonel Steganaliz REST API Servisi."

Bu hedef, 1-2. Hafta'nın CLI-tabanlı steganaliz motorunun sabit bir HTTP sözleşmesi (`POST /api/v1/analyze`) arkasına alınmasıyla (Gün 11-13), ardından bu sözleşmenin nihai yanıt şemasının (`threat_score` dahil) kesinleştirilmesiyle (Gün 14) ve son olarak sözleşmenin her

koşulda (geçerli/geçersiz/aşırı büyük/bozuk girdi) öngörülebilir şekilde davranmasının garanti altına alınmasıyla (Gün 15) adım adım inşa edildi.

3. Kullanılan Teknolojiler

Katman	Araç/Kütüphane
Web çerçevesi	FastAPI, Uvicorn (Gün 11)
Veri doğrulama / şema	Pydantic (<code>HealthResponse</code> , <code>AnalyzeResponse</code>) (Gün 11, 14)
Dosya yükleme	<code>UploadFile</code> , <code>python-multipart</code> (Gün 12)
Asenkron çalıştırma	<code>asyncio.to_thread</code> (Gün 13)
Statik dosya sunumu	<code>StaticFiles</code> (Gün 13)
Hata yönetimi	<code>HTTPException</code> , <code>@app.exception_handler</code> (Gün 15)
Analiz motoru	1-2. Hafta'nın <code>scripts/</code> modülleri (<code>sys.path</code> köprüsüyle, Gün 13)

4. Gün 11 — FastAPI Proje İskeleti ve Pydantic Modelleri

Hedef: Backend servisinin temel iskeletini kurmak: çalışan bir FastAPI uygulaması, yanıt şeması için bir Pydantic model dosyası, `uvicorn` ile ayağa kalkan bir health-check endpoint'i.

`backend/app/` bir Python paketi haline getirildi (`__init__.py`); Gün 13'te `scripts/` modüllerinin buradan import edilmesiyle uyumlu olacak şekilde düşünüldü.

`backend/app/main.py` 'de `FastAPI(...)` uygulaması ve `GET / health-check` endpoint'i (`response_model=HealthResponse`) tanımlandı. `backend/app/models.py` 'ye `HealthResponse` ve — Gün 14'te tam olarak işlenecek — iskelet `AnalyzeResponse` eklendi. Gün 1'de zaten kurulmuş `backend/requirements.txt` (`fastapi`, `uvicorn`, `python-multipart`, `opencv-python`, `Pillow`, `numpy`, `matplotlib`) ek değişiklik gerektirmeden Gün 11'in tüm ihtiyaçlarını karşıladı.

İstek	Sonuç
<code>GET /</code>	<code>{"status":"ok"}</code>
<code>GET /docs</code>	HTTP 200
<code>GET /openapi.json → info</code>	<code>{'title': 'Polyglot / Steganaliz Servisi', 'version': '0.1.0', ...}</code>

5. Gün 12 — Dosya Yükleme Endpoint'i

Hedef: POST /api/v1/analyze ile şüpheli bir görselin yüklenebileceği, yalnızca PNG/JPEG kabul eden, boyut sınırı uygulayan bir giriş noktası yazmak.

Plan yalnızca "MIME type doğrulaması" istiyordu, ama projenin tehdit modeli tam olarak "beyan edilen türün gerçek içerikle uyuşmaması" olduğundan, file.content_type header'ına (client tarafından kolayca sahteleştirilebilir) tek başına güvenilmedi — doğrulama iki aşamalı yapıldı: (1) Content-Type beyaz listede mi, (2) dosyanın ilk baytları (Gün 2'deki PNG/JPEG imzaları) beyan edilen türle eşleşiyor mu. İkisi de tutmazsa 400 . Boyut 25 MB'ı aşarsa 413 . Dosya, path traversal/adı çakışması riskini önlemek için uuid4().hex ile backend/tmp/ altına yazıldı.

Senaryo	Sonuç
Geçerli PNG/JPEG	201, backend/tmp/<uuid>.ext olarak kaydedildi
.txt dosyası, gerçek text/plain	400 "Desteklenmeyen dosya türü..."
.txt dosyası, sahte Content-Type: image/png	400 "magic bytes doğrulaması başarısız"
26 MB sahte PNG	413 "Dosya çok büyük..."

6. Gün 13 — Pipeline'ın FastAPI'ye Asenkron Entegrasyonu

Hedef: 1-2. Hafta'nın scripts/ modüllerini endpoint'e bağlamak; CPU- yoğun işlemleri event loop'u bloklamadan çalıştırmak; ayıklanan videoyu tarayıcıdan erişilebilir kılmak.

scripts/ bir Python paketi olmadığından (modüller birbirini düz isimle import ediyor), 1-2. Hafta'nın test edilmiş CLI davranışını bozmadan kullanmak için backend/app/pipeline.py 'nin başında scripts/ dizini sys.path 'e eklenip modüller olduğu gibi import edildi — script'leri paketleştirmek/taşımak bilinçli olarak bu günün kapsamı dışında bırakıldı.

run_pipeline(saved_path) Gün 4/6/8/9'un adımlarını (trailer → boyut → entropy → varsa extraction → video meta verisi) tek bir sözlükte birleştiriyor. Event loop'u bloklamamak için BackgroundTasks yerine asyncio.to_thread seçildi (yanıtın aynı response'ta, analiz sonucuyla birlikte dönmesi gerektiği için — BackgroundTasks yanıt gönderildikten sonra çalışır ve bu kabul kriterine uymazdı). Ayıklanan video pipeline.MEDIA_DIR e (backend/app/media/) yazılıp app.mount("/media", StaticFiles(...)) ile doğrudan tarayıcıya sunuldu.

Senaryo	Sonuç
Temiz dosya	<code>polyglot_status:false</code> , <code>extracted_video_url:null</code>
Polyglot dosya	<code>polyglot_status:true</code> , <code>extracted_video_url</code> dolu
<code>GET /media/<uuid>_extracted.mp4</code>	200, <code>video/mp4</code> , <code>ffprobe</code> ile açılabilir
5 eşzamanlı analiz sırasında <code>GET /</code>	event loop bloklanmadı (<code>0.014s</code> 'de yanıt)

7. Gün 14 – JSON Yanıt Şemasının Tasarlanması

Hedef: `AnalyzeResponse` 'u plan'ın istediği dört alanla (`polyglot_status` , `threat_score` , `extracted_video_url` , `analysis_summary`) kesinleştirmek; `threat_score` i sinyallerin ağırlıklı birleşimiyle hesaplamak.

`backend/app/pipeline.py` 'ye `compute_threat_score(result)` eklendi. Üç sinyal ağırlıklı toplanıp 100'e kırılıyor:

Sinyal	Kaynak	Ağırlık	Ölçekleme
Trailer tespiti	Gün 4	60	var/yok (0 veya 60)
Entropy farkı	Gün 5	25	<code>delta / 3.0</code> bit , 1.0'de kırılır
Boyut sapması	Gün 6	15	<code>sapma% / 100</code> , 1.0'de kırılır

Ağırlıklar Gün 6/7 raporlarının "tek başına kesin kanıt değil, tamamlayıcı sinyal" disiplinine dayanıyor: trailer tespiti olmadan yalnızca entropy + boyut sapmasıyla skor en fazla 40'a ulaşabiliyor, 100'e değil. `build_analysis_summary` polyglot ise tespit edilen imza/offset/skora Gün 9'un video meta verisini (çözünürlük, süre, codec) de ekleyerek dinamik bir özet cümlesi üretiyor.

Dosya	<code>polyglot_status</code>	<code>threat_score</code>	<code>extracted_video_url</code>
<code>sample.png</code> (temiz)	<code>false</code>	0	<code>null</code>
<code>sample.jpg</code> (temiz)	<code>false</code>	2	<code>null</code>
<code>polyglot_png.png</code>	<code>true</code>	76	dolu
<code>polyglot_jpg.jpg</code>	<code>true</code>	85	dolu

Dosya	polyglot_status	threat_score	extracted_video_url
recompressed_pre_png.png (yeniden sıkıştırılmış polyglot)	true	75	dolu

8. Gün 15 — Swagger Testleri ve Hata Yönetimi

Hedef: API'yi Swagger UI üzerinden uçtan uca doğrulamak; tüm hata senaryolarını anlamlı `HTTPException` 'larla, öngörülmemiş hataları global bir exception handler ile ele almak.

Gün 12/13'te iki hata senaryosu zaten vardı (desteklenmeyen tür, magic bytes uyumsuzluğu). Eksik olanlar tamamlandı: **boş dosya** kontrolü (`400` , önceden yanıltıcı bir mesaj veriyordu), **bozuk dosya** kontrolü (`pipeline.run_pipeline` çağrısı `try/except ValueError` ile sarılıp `422` 'ye çevrildi — önceden yakalanmadığı için düz bir `500` dönüyordu), ve **global** `@app.exception_handler(Exception)` (öngörülmemiş hatalarda stack trace sızdırmadan `500` + sunucu tarafında tam loglama).

Senaryo	Beklenen kod	Sonuç
Swagger'dan 3 farklı dosya (temiz PNG, polyglot PNG, polyglot JPEG)	201	✓
Desteklenmeyen format	400	✓
Boş dosya	400	✓
Bozuk dosya (doğru imza + rastgele veri)	422	✓
Çok büyük dosya	413	✓
Öngörülmemiş hata	500, stack-trace sızmadan	✓ (geçici test satırıyla doğrulanıp geri alındı)

9. Hafta 3 Doğrulama Özeti

Bu raporun hazırlanması sırasında `uvicorn` geçici bir portta (8020) ayağa kaldırılıp Gün 11-15'in tüm senaryoları **canlı sunucuya karşı** yeniden çalıştırıldı ve ilgili günlük rapordaki sonuçlarla birebir eşleştiği bağımsız olarak teyit edildi.

Gün	Doğrulama Yöntemi	Sonuç
11	GET / , GET /docs	Geçti — {"status":"ok"} , HTTP 200
12	.txt yanlış format + sahte Content-Type ile 2 istek	Geçti — ikisi de 400 , mesajlar raporla birebir aynı
13-14	4 dosyayla (sample.png/jpg , polyglot.png.png/jpg) POST /api/v1/analyze	Geçti — threat_score değerleri (0, 2, 76, 85) raporla birebir aynı
15	Boş dosya, bozuk PNG, 26 MB dosya ile 3 istek	Geçti — sırasıyla 400 / 422 / 413 , mesajlar raporla birebir aynı

Test sunucusu ve ürettiği geçici backend/tmp/ / backend/app/media/ dosyaları doğrulama sonrasında temizlendi. Eksik veya açık kalan herhangi bir madde tespit edilmedi.

10. Karşılaşılan Zorluklar ve Öğrenilen Dersler

- **scripts/ 'i paketleştirmeden import etme kararı** (Gün 13): scripts/ modülleri birbirini bare isimle import ettiğinden (paket değil), API katmanından kullanmak için iki seçenek vardı — script'leri paketleştirmek (görelî import'lara çevirmek, CLI kullanımını bozma riski) veya sys.path bootstrap'i (mevcut CLI davranışını korur). İkincisi seçildi; ders: 1-2. Hafta'da test edilmiş bir arayüzü bozmadan yeni bir katman eklemek, "daha temiz" görünen ama regresyon riski taşıyan bir refactor'a tercih edilmeli.
- **BackgroundTasks vs asyncio.to_thread** (Gün 13): İlk bakışta "arka planda çalıştır" ifadesi BackgroundTasks 'ı çağırıyor, ama bu API'nin yanıtı aynı response'ta beklemesi gerektiğinden (kabul kriteri) yanlış araç olurdu — BackgroundTasks yanıt gönderildikten sonra çalışır. Ders: "asenكرون" kelimesi her zaman "arka planda, sonucu beklemeden" anlamına gelmiyor; burada asıl ihtiyaç event loop'u bloklamadan senkron bir sonucu beklemektir.
- **Ağırlıklı threat_score tasarımı Gün 10'dan bilinçli olarak ayrıldı** (Gün 14): Gün 10'un amacı sinyallerin ayrı ayrı başarımını ölçmekti; onları erken birleştirmek o ölçümü bulanıklaştırırdı. Skorlama formülü ancak Gün 6/7'nin "trailer birincil, diğerleri tamamlayıcı" bulgusu netleştikten sonra (Gün 14'te) tasarlandı.
- **Hata yönetiminin geç fark edilen boşluğu** (Gün 15): Gün 13 pipeline entegrasyonu sırasında bozuk bir dosyanın düz bir 500 ile sonuçlandığı fark edilmemişti (mutlu yol testleriyle sınırlı kalınmıştı) — Gün 15'in sistematik hata senaryosu taramasında ortaya çıktı. Ders: her günün "mutlu yol" kabul kriterini geçmesi, olumsuz/ sınır senaryoların da ayrı bir günde (burada Gün 15) sistematik olarak taranmasının yerini tutmuyor.

11. Hafta 3 Çıktısı

Plan'da tanımlanan hedefe ulaşıldı: 1-2. Hafta'nın CLI-tabanlı steganaliz motoru, sabit bir HTTP sözleşmesi (`POST /api/v1/analyze`) arkasında, event loop'u bloklamayan, ayıklanan videoyu statik olarak sunan, nihai bir `threat_score / analysis_summary` üreten ve her hata koşulunda öngörülebilir (400/413/422/500) davranan **dışarıdan sorgulanabilir, tam fonksiyonel bir Steganaliz REST API Servisi** hazır.

12. Sonraki Adımlar (4. Hafta Önizlemesi)

4. Hafta, "Web Dashboard (Frontend) Entegrasyonu, Test & Raporlama" başlığı altında şu adımları içeriyor: sürükle-bırak yükleme arayüzü (Gün 16), frontend-backend entegrasyonu ve sonuç gösterimi (Gün 17), uçtan uca doğrulama testleri (Gün 18) ve dokümantasyon/depo düzenleme (Gün 19). Hafta 4 sonunda tamamlanmış bir web tabanlı tespit paneli ve staj final raporu (Gün 20) hedefleniyor.

13. Ekler — Üretilen Dosyalar

- `docs/gun11-fastapi-iskelet-raporu.md/pdf`
- `docs/gun12-dosya-yukleme-endpoint-raporu.md/pdf`
- `docs/gun13-pipeline-entegrasyonu-raporu.md/pdf`
- `docs/gun14-json-yanit-semasi-raporu.md/pdf`
- `docs/gun15-swagger-hata-yonetimi-raporu.md/pdf`
- `backend/app/main.py` , `backend/app/models.py` , `backend/app/pipeline.py`
- `backend/requirements.txt` (Gün 1'den beri değişmeden doğrulandı)
- `backend/tmp/` , `backend/app/media/` — çalışma zamanında üretilen geçici dosyalar (git'e dahil değil)